Manufacturing Equipment Copilot: Backend and Agent Guide

This document explains the project for a beginner. It describes what happens from the moment a user types a question until the answer appears, why each backend file exists, how BigQuery retrieval works, and how documents become searchable.

1. What the application does

The application is a multimodal retrieval-augmented generation (RAG) assistant for manufacturing manuals. It can answer questions using indexed PDF manuals, return grounded text and tables, show manual images with descriptions, and include relevant manual video or QR links.

The system has four major parts:

1. Frontend: the React interface where the user starts a chat, uploads an image, and reads the response.
2. FastAPI backend: validates requests, manages sessions, runs retrieval, calls the agent, and returns JSON.
3. BigQuery and Cloud Storage: BigQuery stores searchable chunks and related document metadata; Cloud Storage stores processed manual assets.
4. Google models and Document AI: Gemini creates query/document embeddings and generates the final grounded response; Document AI is used offline to inspect QR codes during ingestion.

2. Repository structure

text
Multimodel-RAG-for-Manufacturing-Industry/
??? backend/
?   ??? app/
?   ?   ??? main.py FastAPI application and router setup
?   ?   ??? config.py Environment configuration and validation
?   ?   ??? api/ HTTP endpoints
?   ?   ??? agent/ Planning, Gemini agent, orchestration
?   ?   ??? retrieval/ Embeddings and BigQuery retrieval
?   ?   ??? media/ Images, PDFs, videos, and asset loading
?   ?   ??? responses/ Response schemas and response assembly
?   ?   ??? sessions/ In-memory chat sessions
?   ??? scripts/ Offline QR and connection utilities
?   ??? tests/ Offline backend tests
?   ??? .env Local secrets and settings (never commit)
?   ??? .env.example Safe configuration template
?   ??? requirements.txt Backend dependencies
?   ??? run.cmd Backend launcher
??? frontend/ React/Vite application
??? src/simple_ingestion/ PDF processing and BigQuery loading
??? data/ Local raw/processed data when used
??? terraform/ Infrastructure definitions

3. Starting the application

Start the backend from backend:

powershell
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload

The important detail is app.main:app. The old main:app path is no longer correct because the FastAPI file is inside backend/app.

Start the frontend from frontend:

powershell
npm.cmd run dev

The normal URLs are:

text
Frontend: http://localhost:5173
Backend: http://127.0.0.1:8000
Health: http://127.0.0.1:8000/health

4. Configuration

backend/app/config.py loads backend/.env, validates required settings, and exposes constants used by the rest of the backend. The important settings are:

Setting	Purpose
GOOGLE_CLOUD_PROJECT	Google Cloud project used by Gemini, BigQuery, and storage
BQ_DATASET	BigQuery dataset, currently manufacturing_rag
BQ_LOCATION	BigQuery job location
EMBEDDING_MODEL	Query/document embedding model, currently gemini-embedding-001
EMBEDDING_DIMENSIONS	Shared vector size, currently 768
GCS_BUCKET_NAME	Cloud Storage bucket containing manual assets
REQUEST_TIMEOUT_SECONDS	Maximum time for one chat request
FRONTEND_ORIGIN	Allowed browser origin for CORS

There is deliberately no active single-machine setting. Retrieval reads machine_name from BigQuery, so HSG-2550, HAB-3232, and future machines can coexist.

5. Complete request flow

text
User types question
        ?

?
Frontend sends multipart POST /chat
?
?
chat_routes.py validates session, text, and optional image
?
?
chat_service.py checks inventory requests and calls planner
?
?
planner.py creates a QueryPlan with topic, query, and media needs
?
?
embeddings.py creates a 768-value RETRIEVAL_QUERY vector
?
?
manual_search.py searches document_chunks in BigQuery
?
?
selected chunks are enriched with images, links, tables, sections
?
?
prepare_media.py loads approved assets from Cloud Storage
?
?
agent.py receives only approved evidence and allowed media
?
?
Gemini returns structured AgentAnswer JSON
?
?
response_builder.py validates evidence and assembles final blocks
?
?
FastAPI returns JSON; frontend renders text, tables, images, and videos

6. HTTP API files

backend/app/main.py

Creates the FastAPI application, configures CORS, runs startup checks, and includes the four routers. It is the ASGI entry point used by Uvicorn.

backend/app/api/chat_routes.py

POST /chat is the main endpoint. It:

- loads the session;

- validates a maximum question length;
- accepts PNG, JPEG, or WebP uploads;
- limits uploads to 5 MB;
- normalizes uploaded images;
- calls answer_question with a request timeout;
- records the user and assistant messages;
- converts authentication, model, timeout, and validation failures into useful HTTP errors.

backend/app/api/session_routes.py

Provides POST /sessions, GET /sessions/{id}, and DELETE /sessions/{id}. The frontend creates a session before sending chat messages and uses that ID for the conversation.

backend/app/api/image_routes.py

GET /images/{asset_id} serves an approved image asset when the asset identifier is valid. Images are served through this route rather than exposing arbitrary filesystem paths.

backend/app/api/health_routes.py

GET /health confirms that FastAPI is running and reports the session implementation. It is a process check, not a full BigQuery/Gemini connectivity test.

7. Agent and orchestration files

backend/app/agent/planner.py

The planner makes a small preliminary Gemini call and returns a QueryPlan. It identifies the search query, main topic, follow-up state, whether images/videos/sources are wanted, and whether clarification is needed. It does not answer the technical question.

backend/app/agent/instructions.py

Contains the system rules given to the final agent. The rules require answers to use approved manual evidence, prohibit invented procedures, require evidence IDs for technical blocks, and require image descriptions when images are shown.

backend/app/agent/agent.py

Defines the Google ADK root agent and its model settings. It receives a JSON payload containing the question, workflow state, approved evidence, images, links, and warnings. It returns structured JSON rather than free-form text.

backend/app/agent/chat_service.py

This is the orchestration layer. It connects planning, retrieval, media preparation, ADK execution, validation, and session state.

For inventory questions such as ?which machine documents do you have??, it uses list_available_documents() directly so every distinct machine/document pair is returned. It does not use similarity ranking for that request.

8. Retrieval and BigQuery

backend/app/retrieval/embeddings.py

Creates a cached Vertex AI client and calls gemini-embedding-001 with task type RETRIEVAL_QUERY. The output must contain exactly 768 numbers. A different vector length raises an error because cosine distance requires equal-length vectors.

backend/app/retrieval/bigquery_client.py

Creates a cached BigQuery client, binds query parameters, limits billed bytes, waits for the query, and converts BigQuery rows into normal Python dictionaries. It caches the client connection, not query results.

backend/app/retrieval/manual_search.py

This is the core retriever.

1. It embeds the user question.
2. It extracts up to 12 simple keyword terms.
3. It searches only document_chunks for the initial candidates.
4. It compares the question vector with stored vectors using ML.DISTANCE(..., 'COSINE').
5. It adds a small keyword bonus to the distance score.
6. It keeps the best 10 semantic seed chunks, plus limited media candidates when requested.
7. It expands the selected chunks to the same document/page.
8. It enriches those selected pages from geometry, images, table rows, links, relationships, and sections.
9. It selects up to 24 text chunks, up to 6 image rows, and up to 8 link rows within the response budget.
10. It loads saved QR links for each machine represented in the selected chunks.

The initial retrieval source is always document_chunks. Other tables add context only after a chunk has been selected.

The current vector constraints are intentional:

text
embedding_model gemini-embedding-001
embedding_dimensions 768

Adding more manuals increases the number of rows, not the vector dimension. If a document has another dimension, it must be re-embedded to 768 before loading.

list_available_documents()

This function runs a simple grouped query over document_chunks:

sql

```
SELECT machine_name, document_name
FROM document_chunks
GROUP BY machine_name, document_name
ORDER BY machine_name, document_name
```

It is used for inventory questions and is why the agent can list all machines instead of returning only the nearest HSG chunks.

backend/app/retrieval/qr_link_search.py

Reads verified QR destinations saved in the QR links table. It filters by machine, document, and page so unrelated QR links are not shown.

9. Media and evidence

backend/app/media/prepare_media.py

Takes retrieved chunks and prepares the media payload for the agent. It ensures every supplied image/link retains its source chunk and records warnings instead of silently losing an asset.

backend/app/media/manual_images.py

Loads images from Cloud Storage, renders PDF pages when needed, normalizes uploaded images, and uses bounded caches for repeated assets. It never allows arbitrary user paths.

backend/app/media/video_links.py

Validates URLs and identifies approved manual video targets. Links from unrelated pages are excluded. YouTube links are normalized for frontend playback.

10. Structured response files

backend/app/responses/schemas.py

Defines Pydantic models such as AgentAnswer, AnswerBlock, and ImageDescription. These schemas force the model to return predictable JSON.

backend/app/responses/response_builder.py

Validates evidence IDs, rejects unsupported or invented evidence, preserves all relevant selected media, and converts the model answer into frontend blocks. Image blocks include a description, and source blocks identify the manual, page, and chunk.

11. Sessions

backend/app/sessions/session_store.py

Uses the ADK in-memory session service. A session stores recent messages and workflow state such as the main topic, guided troubleshooting step, and completion status. Sessions expire after the configured TTL, currently up to one hour. Restarting the backend clears in-memory sessions.

12. Ingestion flow

The ingestion code under src/simple_ingestion runs before chat retrieval:

1. pdf_extractor.py extracts PDF text and page elements.
2. table_extractor.py extracts table content.
3. chunk_builder.py creates text, image, table, and link chunks with machine/document/page metadata.
4. relationship_builder.py records relationships between page elements.
5. embedding_generator.py creates RETRIEVAL_DOCUMENT embeddings using 768 dimensions.
6. gcs_ingestor.py uploads processed assets to Cloud Storage.
7. bigquery_loader.py validates vectors and loads the normalized tables into BigQuery.
8. The loader rejects a vector whose declared or actual dimension differs from the shared 768 dimension.
9. pipeline.py coordinates these steps; main.py provides the ingestion CLI.

The machine name is metadata supplied during ingestion. It may be different for every manual, but every document must use the same embedding model and dimension for shared vector search.

13. Document AI and QR workflow

backend/scripts/extract_qr_links.py

Runs offline Document AI processing for a PDF page or image, extracts QR results, validates safe URLs, and writes a reviewable JSON result. It does not modify BigQuery.

backend/scripts/save_qr_links.py

Saves only manually verified QR results to the QR links table. Chat reads these saved links; chat does not call Document AI or write BigQuery data.

backend/scripts/check_connections.py

Performs a small connectivity check for configured Google services.

14. Error handling

- 400: invalid or empty request, unsupported image type, or oversized upload.
- 404: missing session or asset.
- 410: expired or deleted session.
- 502: model returned invalid structured evidence.
- 503: Google authentication, Gemini, BigQuery, or asset service failure.
- 504: request exceeded the backend timeout.

If chat reports service-account impersonation denial, the signed-in Google account needs roles/iam.serviceAccountTokenCreator on the configured service account. This is an IAM issue, not a Uvicorn issue.

15. Useful checks

Check distinct machines:

sql
SELECT DISTINCT machine_name
FROM market-lens-506611.manufacturing_rag.document_chunks
ORDER BY machine_name;

Check embedding compatibility:

sql
SELECT machine_name, embedding_model, embedding_dimensions, COUNT(*) AS chunks
FROM market-lens-506611.manufacturing_rag.document_chunks
GROUP BY machine_name, embedding_model, embedding_dimensions
ORDER BY machine_name;

Every row used in shared retrieval should report gemini-embedding-001 and 768.

Run backend tests:

powershell
cd backend
.\.venv\Scripts\python.exe -m unittest discover -s tests -v

Run frontend checks:

powershell
cd frontend
npm.cmd run lint
npm.cmd run build

16. What is currently implemented

- Multiple machine manuals in one document_chunks table.
- Machine-independent retrieval.
- 768-dimensional Gemini embeddings.
- Semantic cosine search plus keyword scoring.
- Top semantic seeds followed by same-page context expansion.
- Text, table, image, PDF-page, video, and QR-link support.
- Image descriptions in the final answer.
- Evidence validation that rejects unknown or invented citations.
- Session history and guided troubleshooting state.
- CORS, upload validation, bounded timeouts, and service error responses.
- Offline Document AI QR extraction and verified QR-link storage.

- Ten offline backend tests covering API behavior, media completeness, evidence validation, and QR provenance.

The safest mental model is: BigQuery finds approved evidence, Gemini explains only that evidence, and the response builder decides what can be shown to the user.

17. Complete code reference

This section maps each important source file to its responsibility and execution point.

Backend application files

File	Main code	Why it exists	When it runs
app/main.py	Creates FastAPI, CORS, lifespan, and routers	One application entry point	Backend startup
app/config.py	Loads .env, validates project, dataset, bucket, model, dimensions, timeout	Keeps settings in one place and rejects unsafe configuration	Imported during startup
app/api/chat_routes.py	chat() endpoint and exception mapping	Receives questions and uploads and returns HTTP responses	Every chat request
app/api/session_routes.py	Create, read, delete session endpoints	Gives the frontend a conversation ID	Opening, loading, or closing a chat
app/api/image_routes.py	original_image()	Serves approved image assets	Browser requests an image
app/api/health_routes.py	health()	Simple process health check	Browser or operator health check
app/sessions/session_store.py	In-memory entries, locks, TTL, ADK session service	Stores history and guided troubleshooting state	Every session and chat request

Agent files

File	Main code	Why it exists	When it runs
app/agent/planner.py	understand() and cached model client	Converts a user message into QueryPlan	Before normal retrieval
app/agent/instructions.py	Grounding and answer rules	Prevents invented technical information	Sent to the final agent
app/agent/agent.py	ADK root_agent and generation settings	Defines the final structured Gemini agent	After evidence is prepared
app/agent/chat_service.py	answer_question()	Orchestrates planning, retrieval, media, ADK, validation, and state	Every accepted chat message

chat_service.py is the central control file. It first handles inventory questions directly, then calls the planner. It stops early for clarification, out-of-scope requests, or an incompatible model. For a normal question it calls retrieval, prepares assets, sends a compact evidence payload to ADK, validates the returned JSON, builds frontend blocks, and updates the session.

Retrieval files

| File | Main code | Why it exists | When it runs |

|---|---|---|---|
app/retrieval/embeddings.py	embedding_client(), get_embedding()	Creates the 768-number query vector	Normal semantic search
app/retrieval/bigquery_client.py	bq_client(), query()	Runs parameterized BigQuery safely	Every BigQuery operation
app/retrieval/manual_search.py	search_manual(), list_available_documents(), check_tables()	Finds approved chunks and page context	Normal questions and inventory questions
app/retrieval/qr_link_search.py	QR link lookup functions	Reads verified QR destinations for selected pages	Link, video, or QR results

manual_search.py is intentionally the retrieval boundary. The vector seed query reads document_chunks. Related tables are enrichment sources, not independent semantic retrieval sources.

Media files

File	Main code	Why it exists	When it runs
app/media/prepare_media.py	prepare_media()	Selects images and links and collects warnings	After chunks are retrieved
app/media/manual_images.py	Storage client, image loading, PDF rendering, normalization	Loads original manual assets safely	When images or pages are needed
app/media/video_links.py	URL validation and video discovery	Keeps only safe, relevant video links	When links or videos are requested

The machine_name on each retrieved row is passed into asset loading. This is required because multiple machines can have documents with similar names.

Response files

File	Main code	Why it exists	When it runs
app/responses/schemas.py	QueryPlan, AgentAnswer, AnswerBlock, media schemas	Defines the allowed JSON shape	Planner and final-agent validation
app/responses/response_builder.py	text_response(), final_blocks()	Converts validated agent output into frontend blocks	Before the HTTP response

response_builder.py is the final safety boundary. It verifies that every evidence ID came from the approved bundle, preserves selected media, and rejects unsupported or invented evidence.

18. Frontend code map

Area	Responsibility	Execution point
frontend/src/App.jsx	Page-level chat layout and session lifecycle	Browser startup
frontend/src/main.jsx	React root mounting and global styles	Browser startup
frontend/src/components/chat/ChatInput.jsx	Text entry, image selection, send action	User input
frontend/src/components/chat/ChatThread.jsx	Conversation list and message order	Every render
frontend/src/components/chat/ChatMessage.jsx	One user or assistant message	Every message

frontend/src/components/chat/ResponseRenderer.jsx	Text, table, image, link, warning rendering	Assistant response
frontend/src/components/chat/ImageUpload.jsx	Image validation and preview	Optional upload
frontend/src/components/layout/Header.jsx	Title and New Chat control	Page layout
frontend/src/components/layout/Welcome.jsx	Empty-state examples	New session
frontend/src/components/layout/GearLogo.jsx	Product mark	Header and welcome view
frontend/src/hooks/useChat.js	Chat state and send lifecycle	App initialization and messages
frontend/src/services/chatService.js	Session, chat, history, and image HTTP calls	Every frontend/backend request
frontend/src/config/appConfig.js	API base URL and endpoint names	Frontend startup
frontend/src/utils/validateImage.js	Client-side image type and size checks	Before upload
frontend/src/styles/*	Layout and visual appearance	Browser rendering

The frontend does not call BigQuery, Gemini, or Cloud Storage directly. It only calls FastAPI and renders the JSON that FastAPI returns.

19. Ingestion code map

File	Responsibility	When used
src/simple_ingestion/main.py	Command-line entry point	Starting ingestion manually
src/simple_ingestion/pipeline.py	Coordinates extraction, chunking, embeddings, upload, and load	Full manual ingestion
src/simple_ingestion/pdf_extractor.py	Reads PDF pages and elements	First extraction step
src/simple_ingestion/table_extractor.py	Converts detected tables into rows	During extraction
src/simple_ingestion/chunk_builder.py	Builds normalized text/image/table/link chunks	After extraction
src/simple_ingestion/relationship_builder.py	Builds source/target element relationships	After chunk construction
src/simple_ingestion/embedding_generator.py	Calls RETRIEVAL_DOCUMENT embeddings	Before BigQuery load
src/simple_ingestion/gcs_ingestor.py	Uploads processed assets	Before or during load
src/simple_ingestion/bigquery_loader.py	Validates and writes normalized rows to all tables	Final ingestion step
src/simple_ingestion/config.py	Ingestion project, bucket, dataset, and dimensions	Ingestion startup
src/simple_ingestion/test_normalization.py	Tests ingestion normalization	Ingestion development checks
src/simple_ingestion/inspect_unresolved.py	Diagnostic relationship inspection	Manual debugging only
src/simple_ingestion/validate_relationship_data.py	Validates relationship artifacts	Manual debugging only

The loader rejects a vector if its declared dimension differs from its actual array length or from the shared configured dimension of 768. This prevents unusable data from reaching the shared vector-search table.

20. Utility scripts and tests

File	Purpose	When used
backend/scripts/extract_qr_links.py	Document AI QR extraction and safe URL output	Offline manual inspection
backend/scripts/save_qr_links.py	Saves manually verified QR links	After human verification
backend/scripts/check_connections.py	Checks configured Google services	Troubleshooting
backend/tests/test_api.py	Health, CORS, sessions, uploads, timeout and model errors	Backend regression tests
backend/tests/test_response_builder.py	Evidence, media, image-description, and video filtering	Response regression tests
backend/tests/test_documentai_qr.py	QR provenance, deduplication, and unsafe URL checks	QR regression tests

21. One complete code flow

A normal text question

Example: What is the saw blade speed range for HAB-3232?

1. ChatInput.jsx collects the text.
2. useChat.js calls chatService.sendChat().
3. chatService.js sends multipart form data to /chat.
4. chat_routes.chat() validates the session and text length.
5. session_store.py loads the session and locks it so two requests cannot modify it at once.
6. chat_service.answer_question() calls planner.understand().
7. planner.py asks Gemini for a QueryPlan.
8. embeddings.get_embedding() creates the 768-value query vector.
9. manual_search.search_manual() runs the fixed parameterized query against document_chunks.
10. BigQuery ranks rows by cosine distance plus keyword hits.
11. Explicit machine names cause per-machine candidate retention, so one machine cannot crowd out another.
12. manual_search.py expands selected chunks to their document pages and enriches approved context.
13. prepare_media.py loads requested images, PDF pages, and links.
14. chat_service.py converts long internal chunk IDs to short evidence IDs.
15. agent.py sends the question and approved evidence to ADK.
16. instructions.py tells Gemini to answer every distinct question and avoid unsupported claims.
17. Gemini returns AgentAnswer JSON.
18. schemas.py validates the JSON.
19. chat_service.py maps short evidence IDs back to real chunk IDs.
20. response_builder.py creates text/table/image/link blocks and adds descriptions.
21. session_store.py records the user and assistant messages.
22. chat_routes.py returns JSON.
23. ChatThread.jsx, ChatMessage.jsx, and ResponseRenderer.jsx display the answer.

A machine-document inventory question

Example: Which machine documents do you have?

1. The frontend sends the normal /chat request.
2. chat_service.py recognizes the inventory wording.
3. list_available_documents() runs a grouped query over document_chunks.
4. Results are grouped by machine_name in Python.
5. text_response() creates a direct answer.
6. The session records the response without calling the model or vector search.

A question asking for an image

1. Normal planning and document_chunks retrieval occur first.
2. The planner sets wants_visual=true.
3. manual_search.py keeps image candidates and geometry for selected pages.
4. prepare_media.py calls read_image() or render_page() with the row's machine name.
5. The image bytes are sent to the agent and later returned as an approved asset.
6. ResponseRenderer.jsx displays the image, caption, and detailed description.

A question asking for video or QR links

1. Normal chunk retrieval identifies relevant document pages.
2. document_links and the QR links table are restricted to those pages.
3. video_links.py validates and resolves relevant video targets.
4. response_builder.py keeps only links supported by the selected evidence.
5. The frontend renders links or embedded video targets.

A two-machine comparison

1. The planner receives both machine names.
2. manual_search.py extracts identifiers such as HAB-3232 and HSG-2550.
3. BigQuery keeps up to 10 relevant seed chunks per mentioned machine.
4. Same-page enrichment runs for the selected chunks.
5. The final agent receives evidence from both machines and is instructed to answer every distinct part.
6. If either manual lacks the requested specification, that part is reported as unavailable rather than invented.

22. Why a model error can occur after successful retrieval

BigQuery retrieval and Gemini answer generation are separate operations. A request can successfully retrieve chunks and still fail later when ADK calls Gemini. A 504 DEADLINE_EXCEEDED from the final model call means the model did not finish the structured response in time, often because the evidence payload contains too much text, image data, or metadata. The backend therefore bounds the evidence context and output size. A 503 shown by the frontend is the safe HTTP wrapper around that provider failure; the backend terminal contains the underlying Google error.

23. Full system architecture flow

A. Manual ingestion flow

text

PDF/manual
  -> pdf_extractor.py and table_extractor.py
  -> chunk_builder.py creates document chunks
  -> relationship_builder.py creates element relationships
  -> embedding_generator.py creates 768-value document embeddings
  -> gcs_ingestor.py uploads PDFs/images to Cloud Storage
  -> bigquery_loader.py validates and writes BigQuery tables

document_chunks becomes the searchable table. The related tables store extra information connected by chunk_id, source_id, document_name, and page number.

B. Normal chat flow

text
1. User types question
2. ChatInput.jsx collects it
3. useChat.js calls chatService.js
4. chatService.js sends POST /chat
5. chat_routes.py validates request and session
6. session_store.py locks the conversation
7. chat_service.py calls planner.py
8. planner.py returns QueryPlan
9. embeddings.py creates the 768-value query vector
10. manual_search.py searches document_chunks
11. BigQuery ranks cosine distance plus keyword hits
12. Selected chunks are expanded to their document/page
13. Relevant related tables are joined for enrichment
14. prepare_media.py loads selected images, pages, and links
15. chat_service.py builds a compact approved-evidence payload
16. agent.py sends it to the ADK root agent
17. instructions.py enforces grounding and complete answers
18. Gemini returns AgentAnswer JSON
19. schemas.py validates the JSON
20. Evidence IDs are checked against retrieved chunks
21. response_builder.py creates final text/table/image/link blocks
22. session_store.py saves conversation history
23. FastAPI returns JSON
24. ResponseRenderer.jsx displays the answer

C. Retrieval decision flow

text
Question
  |
  +-- inventory question? -> GROUP BY machine_name, document_name in document_chunks
  |

+-- normal question -> embed question -> search document_chunks
|
+-- mentions two machines -> keep top candidates separately per machine
|
+-- asks image -> add document_images and geometry for selected chunks
|
+-- asks table/specification -> add document_table_rows
|
+-- asks video/link/QR -> add document_links and verified QR links
|
+-- asks section context -> add document_sections and relationships

D. Exact table responsibility

Table	Used for	Retrieval role
document_chunks	Text, machine, document, page, embeddings	Only initial semantic retrieval and inventory
document_images	Image IDs and storage paths	Enrichment after a chunk is selected
document_chunk_geometry	Coordinates on a page	Image/page positioning
document_table_rows	Extracted table rows	Specification/table answers
document_links	URLs embedded in manuals	Link and video answers
chunk_relationships	Source/target element links	Related page context
document_sections	Titles and hierarchy	Section context
section_elements	Section membership	Section expansion
QR links table	Verified QR destinations	QR results for selected pages

E. Complete file sequence for a technical question

ChatInput.jsx -> useChat.js -> chatService.js -> chat_routes.py -> session_store.py -> chat_service.py -> planner.py -> embeddings.py -> bigquery_client.py and manual_search.py -> optional qr_link_search.py -> prepare_media.py -> manual_images.py and video_links.py -> agent.py and instructions.py -> schemas.py -> response_builder.py -> chat_routes.py -> ResponseRenderer.jsx.

F. What happens when something fails

text
Invalid input -> 400 response from chat_routes.py
Missing session -> 404 response from session_routes.py
Expired session -> 410 response from chat_routes.py
Gemini busy -> 503 response with retry message
Gemini deadline -> 503 response; backend terminal shows 504
Invalid agent JSON -> 502 response from evidence validation
BigQuery failure -> 503 response; backend terminal contains query error
Asset unavailable -> response continues with asset warning

24. What was implemented for this project

The code was modularized without changing the intended application behavior. Backend responsibilities were separated into API, agent, retrieval, media, responses, sessions, and scripts. Frontend components were separated into chat, layout, hooks, services, configuration, and utilities. BigQuery retrieval now supports multiple machine names, uses 768-dimensional embeddings, retains evidence per explicitly mentioned machine, provides a direct all-document inventory query, and passes machine metadata into media loading. The agent is instructed to answer every distinct question in a message. Context size is bounded to reduce Gemini deadline failures, while evidence validation keeps answers grounded.